

Explorando la Comunicación RS232 con Raspberry Pi Pico 2W

Universidad Militar Nueva Granada – Ingeniería en Telecomunicaciones
Miguel Ángel Plazas – Daniel Mateo Alegría Bernate
Docente: José de Jesús Rúgeles Uribe

Resumen - En esta práctica se estudia el funcionamiento de la comunicación serial RS232 mediante el uso de una Raspberry Pi Pico 2W, MicroPython y un osciloscopio digital. Se analiza la relación existente entre la tasa de transmisión, expresada en baudios, y el tiempo de duración de cada bit, verificando experimentalmente que el tiempo de bit corresponde al inverso de la tasa de transmisión. Además, se estudia la estructura de las tramas seriales, identificando el bit de inicio, los bits de datos, el bit de paridad y los bits de parada.

A partir de diferentes configuraciones de velocidad, número de bits de datos y paridad, se observan las variaciones producidas en las señales transmitidas. También se mide el tiempo total necesario para transmitir caracteres y tramas completas, comparando los valores teóricos con las mediciones obtenidas mediante el osciloscopio. Finalmente, se plantea la interconexión de dos Raspberry Pi Pico 2W para implementar transmisión y recepción de datos, incluyendo un mecanismo de confirmación mediante ACK y la transferencia de archivos.

Palabras clave - Bit de paridad, Comunicación serial, MicroPython, Protocolo RS232, Raspberry Pi Pico 2W, Tasa de baudios, Trama UART..

Abstract — This lab explores the operation of RS232 serial communication using a Raspberry Pi Pico 2W, MicroPython, and a digital oscilloscope. The relationship between the transmission rate, expressed in baud, and the bit duration is analyzed, experimentally verifying that the bit duration is the inverse of the transmission rate. The structure of serial frames is also studied, identifying the start bit, data bits, parity bit, and stop bits.

Using different configurations of speed, number of data bits, and parity, the resulting variations in the transmitted signals are observed. The total time required to transmit characters and complete frames is measured, comparing the theoretical values with the measurements obtained using the oscilloscope. Finally, the interconnection of two Raspberry Pi Pico 2Ws is proposed to implement data transmission and reception, including an acknowledgment mechanism using ACK and file transfer.

Keywords— Baud rate, Frame, Parity bit, RS232 protocol, Raspberry Pi Pico 2W, Serial communication, UART.

I. INTRODUCCIÓN

La comunicación serial permite transmitir información digital entre dispositivos mediante el envío secuencial de bits a través de una línea de comunicación. Uno de los mecanismos empleados para este tipo de transmisión es RS232, cuyo funcionamiento puede analizarse mediante parámetros como la tasa de baudios, el tiempo de bit, el número de bits de datos, la paridad y los bits de parada.

En esta práctica se emplea una Raspberry Pi Pico 2W para generar señales seriales utilizando la interfaz UART y MicroPython. La señal transmitida es observada mediante un osciloscopio digital con el propósito de identificar sus características, su tiempo de bit para diferentes tasas de transmisión, verificando experimentalmente la relación entre ambas magnitudes, el envío de tramas de mayor longitud la estructura de las tramas RS232 mediante diferentes combinaciones de velocidad, bits de datos y paridad.

Esto permite identificar cada uno de los elementos que conforman una transmisión serial y observar cómo las modificaciones en los parámetros de configuración afectan la duración y estructura de la señal.

II. MARCO TEÓRICO

A. Comunicación serial UART

Método de transmisión en el que los bits se envían secuencialmente a través de una línea. En una comunicación UART (Universal Asynchronous Receiver-Transmitter), los dispositivos deben compartir parámetros como la velocidad de transmisión, número de bits de datos, paridad y bits de parada [1], [2].

B. Raspberry Pi Pico 2W

La Raspberry Pi Pico 2W es una placa de desarrollo basada en el microcontrolador RP2350, diseñada para aplicaciones de sistemas embebidos, automatización y comunicación

inalámbrica. Cuenta con interfaces de comunicación como UART, I2C y SPI, además de conectividad Wi-Fi y Bluetooth, lo que permite su integración con diferentes sensores, módulos y dispositivos externos. Su tamaño compacto y bajo consumo la hacen adecuada para proyectos de adquisición, procesamiento y transmisión de datos. En este proyecto, la Raspberry Pi Pico 2W se emplea como plataforma de control y comunicación entre los diferentes componentes del sistema. [5]

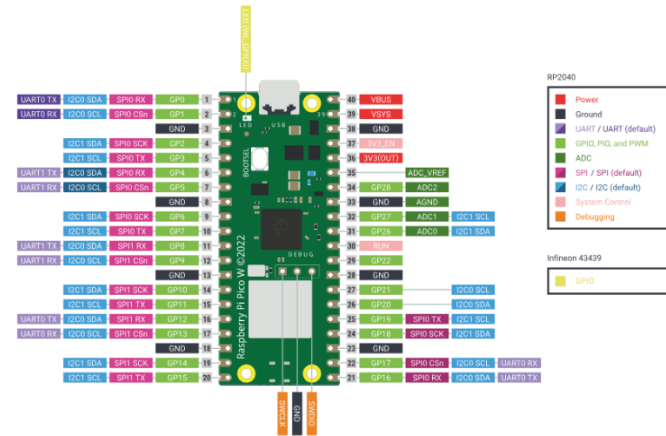


Figure 1. Raspberry Pi Pico 2W

C. Tasa de transmisión o baudios

El espectrograma representa la evolución temporal de La tasa de transmisión indica la cantidad de bits transmitidos por segundo cuando se considera una transmisión serial donde cada símbolo representa un bit. [2].

$$T_b = \frac{1}{\text{Baudios}}$$

D. Trama de comunicación serial

Una transmisión serial no está formada únicamente por los bits correspondientes al carácter. [2], [3]. La información se organiza en una trama que puede contener:

1. Bit de inicio.
2. Bits de datos.
3. Bit de paridad (cuando está configurado)
4. Bits de parada.

El número total de bits de una trama puede expresarse como:

$$N_{trama} = inicio + datos + Paridad + Parada$$

Además, en una transmisión UART convencional los bits de datos se transmiten comenzando por el bit menos significativo (LSB) [2].

E. Bits de datos, LSB, MSB y código ASCII

Los caracteres transmitidos pueden representarse mediante el código ASCII. Cada carácter posee un valor numérico que puede expresarse en forma binaria. [4].

Para una palabra de 8 bits:

$$D = d_7 d_6 d_5 d_4 d_3 d_2 d_1 d_0$$

Donde d_0 es el bit menos significativo (LSB) y d_7 el bit más significativo (MSB). En UART, los bits de datos se transmiten normalmente comenzando por el LSB.

F. Tiempo de transmisión de un carácter

El tiempo necesario para transmitir un carácter depende de la cantidad de bits de su trama y de la duración de cada bit. [2]

$$T_{Caracter} = N_{trama} \times T_b$$

G. Tiempo total de transmisión

Para transmitir varios caracteres, el tiempo total puede calcularse mediante:

H. Paridad

El bit de paridad permite detectar determinados errores en la transmisión de datos. Puede configurarse como paridad par, impar o deshabilitada. [2], [3]

Cuando se utiliza paridad, se agrega un bit adicional a la trama. Por ejemplo, con 8 bits de datos, un bit de inicio y un bit de parada:

- Sin paridad: 10 bits por trama.
- Con paridad: 11 bits por trama.

Por lo tanto, la configuración de paridad modifica tanto la estructura como el tiempo de transmisión de la trama.

I. Transmisión y recepción mediante UART

La comunicación entre dos Raspberry Pi Pico 2W se realiza conectando la salida TX de un dispositivo con la entrada RX del otro:

$$\begin{aligned} TX_1 &\rightarrow RX_2 \\ TX_2 &\rightarrow RX_1 \end{aligned}$$

Además, ambos dispositivos deben compartir una referencia común de tierra:

$GND1 \rightarrow GND2$

De esta manera se puede realizar una comunicación bidireccional, en la que un dispositivo transmite un carácter y el otro lo recibe y puede responder mediante un carácter de confirmación (ACK). [2], [5].

III. PROCEDIMIENTO EXPERIMENTAL

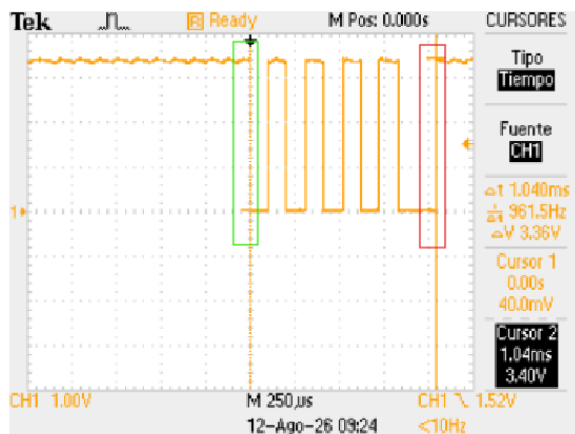
A. Configuración inicial

Se programó la Raspberry Pi Pico 2W en MicroPython, configurando el periférico UART0 con el pin GP0 como terminal TX y GND (pin 3) como referencia común con la punta de tierra del osciloscopio. La punta activa del canal 1 (CH1) del osciloscopio se conectó directamente al terminal TX (pin 1) del dispositivo.

Se empleó el osciloscopio digital Tektronix en modo de disparo por flanco sobre CH1, ajustando la base de tiempo y la escala vertical según la tasa de baudios bajo prueba.

B. Envío de caracteres ASCII

Como primera prueba se ejecutó el programa base de la guía (ANEXO A). El cual transmite de forma periódica el carácter 'U' (85 decimal) a 9600 baudios, 8 bits de datos y sin paridad.



TDS 2012B - 9:28:56 a. m. 12/08/2026

Figure 2. Trama completa del carácter 'U' a 9600 baudios

se observa una trama completa correspondiente al carácter "U". La transmisión inicia con un bit de inicio en nivel bajo(verde),

seguido por los ocho bits de datos y finalmente un bit de parada en nivel alto(rojo)

la trama está compuesta por un total de:

$$N_{trama} = 1 + 8 + 0 + 1 = 10 \text{ bits}$$

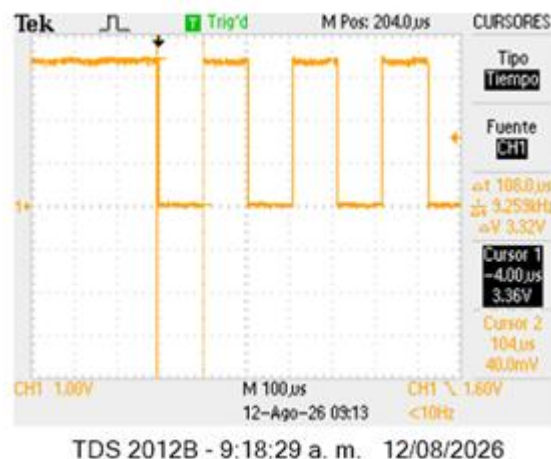


Figure 3. Medición del tiempo de un bit.

se realizó la medición de la duración de un bit mediante los cursores del osciloscopio. El valor experimental obtenido fue aproximadamente de 108 µs

Para determinar el valor teórico se utiliza la relación entre la tasa de transmisión y el tiempo de bit:

$$T_{b\text{ Teo}} = \frac{1}{9600} = 104,17 \mu s$$

$$t_b = \frac{[104.17 - 108]}{104.17} \times 100\% = -3.67\%$$

Parámetro	Valor
Tasa de transmisión	9600 baudios
Tiempo de bit teórico	104,17 µs
Tiempo de bit experimental	108 µs
Diferencia	4,83 µs
Error porcentual	-3.67%

Tabla 1. Resultados medición tiempo de bit 'U' a 9600 baudios.

El resultado experimental presenta una diferencia pequeña respecto al valor teórico, con un error aproximado del -3,67 %, lo que indica que la tasa de transmisión configurada en la corresponde adecuadamente a los 9600 baudios establecidos.

IV. MEDICIÓN DE TIEMPOS DE BITS

Se transmitió el carácter 'Y' modificando únicamente la tasa de baudios (parámetros bits=8, parity=None fijos) y se midió el tiempo de un bit individual con los cursores de tiempo del osciloscopio, así como los niveles de tensión máximo y mínimo con cursores de amplitud. Puede ver los códigos utilizados para cada caso en el (ANEXO B).

Cálculos

A 1200 baudios

$$tb = \frac{1}{1200} = 833.33 \mu s$$

$$\%error = \frac{|836.00 - 833.33|}{833.33} \times 100 = 0.32 \%$$

A 4800 baudios

$$tb = \frac{1}{4800} = 208.33 \mu s$$

$$\%error = \frac{|207.00 - 208.33|}{208.33} \times 100 = 0.64 \%$$

A 9600 baudios

$$tb = \frac{1}{9600} = 104.17 \mu s$$

$$\%error = \frac{|105.00 - 104.17|}{104.17} \times 100 = 0.80 \%$$

A 38400 baudios

$$tb = \frac{1}{38400} = 26.04 \mu s$$

$$\%error = \frac{|26.20 - 26.04|}{26.04} \times 100 = 0.61 \%$$

A 57600 baudios

$$tb = \frac{1}{57600} = 17.36 \mu s$$

$$\%error = \frac{|17.50 - 17.36|}{17.36} \times 100 = 0.80 \%$$

A 115200 baudios

$$tb = \frac{1}{115200} = 8.68 \mu s$$

$$\%error = \frac{|8.80 - 8.68|}{8.68} \times 100 = 1.38 \%$$

Una vez realizados los cálculos se procedió a comparar los datos teóricos con los experimentales resultando en la siguiente tabla

COMPARACIÓN DE RESULTADOS

Una vez realizados los cálculos teóricos y obtenidas las mediciones experimentales, se organizaron los resultados en la Tabla I.

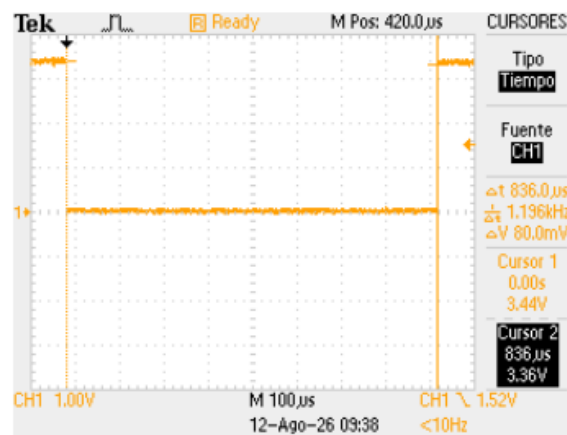
Baudios	tb teórico (μs)	tb experim. (μs)	Δt (μs)	% error	V _{má} x (V)	V _{mín} (V)
1200	833.33	836.0	+2.67	0.32%	3.40	0.00
4800	208.33	207.0	-1.33	0.64%	3.40	0.00
9600	104.17	105.0	+0.83	0.80%	3.40	0.00
38400	26.04	26.20	+0.16	0.61%	3.40	0.00
57600	17.36	17.50	+0.14	0.80%	3.40	0.00
115200	8.68	8.80	+0.12	1.38%	3.40	0.00

Tabla 2. Medición tiempos bit carácter 'Y' a diferentes tasas de baudios.

Podemos observar que el tiempo de bit disminuye conforme aumenta la tasa de transmisión, comportamiento que coincide con la relación teórica. El mayor tiempo de bit se obtuvo a 1200 baudios, con un valor experimental de 836 μs, mientras que el menor se obtuvo a 115200 baudios, con 8,80 μs.

Los porcentajes de error se encuentran entre 0,32 % y 1,38 %, por lo que las mediciones presentan una buena aproximación respecto a los valores teóricos

En cuanto a los niveles de tensión, se obtuvo un valor máximo de 3,40 V y un valor mínimo de 0,00 V para todas las configuraciones analizadas, indicando que la modificación de la tasa de transmisión afectó principalmente la duración temporal de los bits, mientras que los niveles de tensión observados permanecieron constantes.



TDS 1012B - 9:42:52 a. m. 12/08/2026

Figure 4. Bit a 1 200 baudios (Δt = 836.0 μs)

La duración experimental del bit es 836 μs frente a 833.33 μs teóricos, con un error de 0.32 %. La diferencia es pequeña frente

al intervalo total medido. lo que demuestra una relación similar entre la configuración de 1200 baudios y la duración temporal observada.



Figure 5. Bit a 4800 baudios ($\Delta t = 207.0 \mu s$)

La duración experimental del bit es 207 μs frente a 208.33 μs teóricos, con un error de 0.64 %. La diferencia es pequeña frente al intervalo total medido



Figure 6 Bit a 9 600 baudios ($\Delta t = 105.0 \mu s$)

Para 9600 baudios se obtuvo experimentalmente un tiempo de bit de 105,0 μs , frente a los 104,17 μs calculados teóricamente. El error de 0.80 % confirma la relación inversa entre baudios y tiempo de bit.

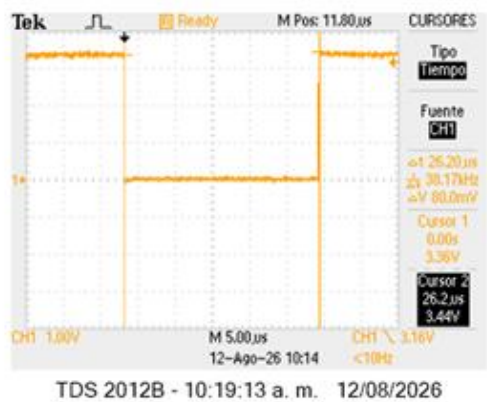


Figure 7.Bit a 38 400 baudios ($\Delta t = 26.20 \mu s$).

La medición experimental fue de 26,20 μs , mientras que el valor teórico corresponde a 26,04 μs . La diferencia de 0,16 μs produce un error de 0,61 %, mostrando una buena aproximación entre la medición y el comportamiento teórico.



Figure 8.Bit a 57 600 baudios ($\Delta t = 17.50 \mu s$).

Para 57600 baudios se obtuvo un tiempo experimental de 17,50 μs , cercano al valor teórico de 17,36 μs . El error de 0,80 % evidencia nuevamente una correspondencia adecuada entre ambos valores.



Figure 9.Bit a 115 200 baudios ($\Delta t = 8.800 \mu s$)

En la configuración de 115200 baudios se obtuvo un tiempo experimental de 8,80 μs , frente a los 8,68 μs teóricos. El error fue de 1,38 %, siendo el mayor de las mediciones realizadas.

Esto puede relacionarse con la menor duración del intervalo medido, donde pequeñas variaciones en la lectura del osciloscopio tienen un efecto relativo mayor.

V. ANÁLISIS DE LA ESTRUCTURA DE LAS TRAMAS Y EFECTO DE LA PARIDAD

Se transmitieron cinco caracteres ASCII distintos al carácter 'Y' empleado en la sección anterior, cada uno bajo una combinación diferente de baudios, bits de datos y paridad.

Para cada carácter se midió el tiempo total de trama en al menos dos configuraciones (con y sin paridad, o dos variantes de

paridad) con el fin de evidenciar el efecto de este parámetro sobre la duración de la trama. Puede ver los códigos utilizados para cada caso en el (ANEXO C).

Podemos observar 7 bits de datos, paridad y un bit de parada. El tiempo de 8.200 ms se aproxima a 8.333 ms teóricos para 10 bits totales, con 1.60 % de error. En la trama se pueden delimitar inicio, datos, paridad y parada.

#	Carácter	Decimal	Binario (7/8)	Baudios	Bits datos
1	?	63	0111111	1200	7
2	\$	36	00100100	57600	8
3	B	66	01000010	4800	8
4	v	118	01110110	9600	8
5	-	45	0101101	115200	7

Tabla 3. Caracteres ASCII empleados en el análisis de trama.

A. Carácter 1: '?'

Valor decimal: 63. Valor binario (7 bits): 0111111. Orden de transmisión (LSB primero): 1,1,1,1,1,1,0. Configuración UART: 1200 baudios, 7 bits de datos. Bit de inicio: nivel bajo (0) previo al primer bit de dato. Bit(s) de parada: nivel alto (1) al final de la trama.

Configuración	Tiempo total Teó	Tiempo total Exp	%
Sin paridad (9 bits)	7.500 ms	7.500 ms	0.00%
Con paridad, original(10 bits)	8.333 ms	8.200 ms	1.60%
Con paridad, modificada (10 bits)	8.333 ms	8.300 ms	0.40%

Tabla 4. Tiempo total de trama para el carácter '?' (1200 baudios, 7 bits de datos).

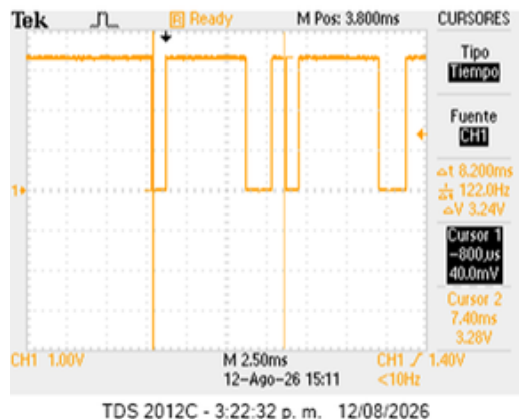


Figure 10. Carácter '?' con paridad, prueba original ($\Delta t = 8.200$ ms)

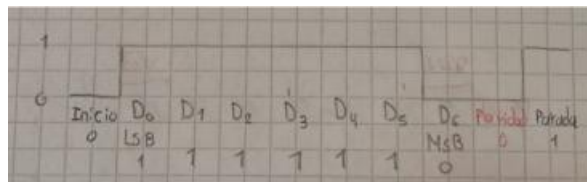


Figure 11. Dibujo Trama "?"

Segmento	Valor	Intervalo teórico
Bit de inicio (start)	0 (bajo)	0.00 μ s – 833.33 μ s
Dato d0 (LSB)	1	833.33 μ s – 1666.66 μ s
Dato d1	1	1666.66 μ s – 2499.99 μ s
Dato d2	1	2499.99 μ s – 3333.32 μ s
Dato d3	1	3333.32 μ s – 4166.65 μ s
Dato d4	1	4166.65 μ s – 4999.98 μ s
Dato d5	1	4999.98 μ s – 5833.31 μ s
Dato d6 (MSB)	0	5833.31 μ s – 6666.64 μ s
Paridad (tipo no confirmado en la captura)	par $\rightarrow$ 0 / impar $\rightarrow$ 1	6666.64 μ s – 7499.97 μ s
Bit de parada (stop)	1 (alto)	7499.97 μ s – 8333.30 μ s

Tabla 5. Estructura de bits de la trama del carácter '?' con paridad.

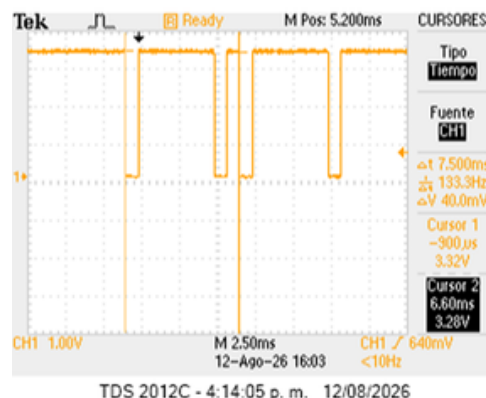


Figure 12. Carácter '?' sin paridad ($\Delta t = 7.500$ ms, 1200 baudios).

Al retirar el bit de paridad la trama pasa de 10 a 9 bits. La duración medida de 7.500 ms coincide con la teórica, por lo que el error es 0.00 %. La comparación con la Fig. 8 evidencia directamente el efecto temporal de la paridad.

Segmento	Valor	Intervalo teórico
Bit de inicio (start)	0 (bajo)	0.00 μ s – 833.33 μ s
Dato d0 (LSB)	1	833.33 μ s – 1666.66 μ s
Dato d1	1	1666.66 μ s – 2499.99 μ s
Dato d2	1	2499.99 μ s – 3333.32 μ s
Dato d3	1	3333.32 μ s – 4166.65 μ s
Dato d4	1	4166.65 μ s – 4999.98 μ s
Dato d5	1	4999.98 μ s – 5833.31 μ s
Dato d6 (MSB)	0	5833.31 μ s – 6666.64 μ s
Bit de parada (stop)	1 (alto)	6666.64 μ s – 7499.97 μ s

Tabla 6. Estructura de bits de la trama del carácter '?' sin paridad.

al remover la paridad la trama pasa de 10 a 9 bits y el tiempo total disminuye =833 μ s, coherente con la diferencia medida (700–800 μ s) entre las tramas con y sin paridad.

Las dos variantes 'con paridad' (par e impar) muestran duración total casi idéntica (8.200 ms vs. 8.300 ms), confirmando que el tipo de paridad no altera el número de bits ni, por tanto, el tiempo de transmisión

B. Carácter 2: '\$'

Valor decimal: 36. Valor binario (8 bits): 00100100. Orden de transmisión (LSB primero): 0,0,1,0,0,1,0,0. Configuración UART: 57600 baudios, 8 bits de datos. Bit de inicio: nivel bajo (0) previo al primer bit de dato. Bit(s) de parada: nivel alto (1) al final de la trama.

Configuración	Tiempo total teó	Tiempo total exp	%
Con paridad (11 bits: 1+8+1+1, inferido)	190.97 μ s	192.0 μ s	0.54%

Tabla 7. Tiempo total de trama para el carácter '\$' (57600 baudios, 8 bits de datos).

Cálculos

$$tb = \frac{1}{57600} = 17.36 \mu s$$

$$\text{Con paridad: nbits} = 1 + 8 + 1 + 1 = 11$$

$$T_{trama} = 11 \times 17.36 = 190.97 \mu s$$

$$\%E = \frac{|192.00 - 190.97|}{190.97} \times 100 = 0.54 \%$$

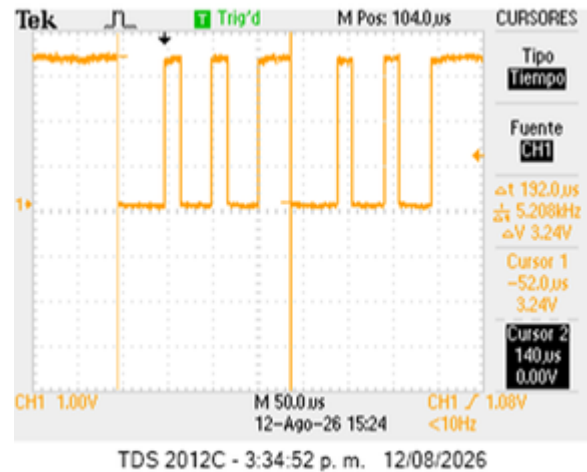


Figure 13. Carácter '\$' tiempo total de trama ($\Delta t = 192.0 \mu$ s)

La duración total medida para el carácter \$ es 192.0 μ s, frente a 190.97 μ s teóricos, con 0.54 % de error. La configuración utilizada es compatible con una trama de 11 bits y permite analizar inicio, datos, paridad y parada.

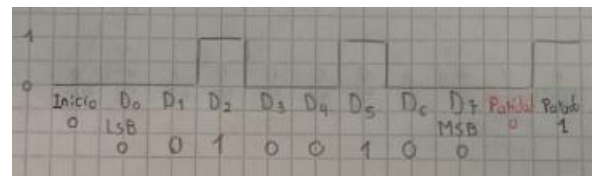


Figure 14. Dibujo Trama "\$"

Segmento	Valor	Intervalo teórico
Bit de inicio (start)	0 (bajo)	0.00 μ s – 17.36 μ s
Dato d0 (LSB)	0	17.36 μ s – 34.72 μ s
Dato d1	0	34.72 μ s – 52.08 μ s
Dato d2	1	52.08 μ s – 69.44 μ s
Dato d3	0	69.44 μ s – 86.80 μ s
Dato d4	0	86.80 μ s – 104.16 μ s

Dato d5	1	104.16 μ s – 121.52 μ s
Dato d6	0	121.52 μ s – 138.88 μ s
Dato d7 (MSB)	0	138.88 μ s – 156.24 μ s
Paridad (tipo no confirmado en la captura)	par→0	156.24 μ s – 173.60 μ s
Bit de parada (stop)	1 (alto)	173.60s – 190.96 μ s

Tabla 8. Estructura de bits de la trama del carácter '\$' con paridad.

C. Carácter 3: 'B'

Valor decimal: 66. Valor binario (8 bits): 01000010. Orden de transmisión (LSB primero): 0,1,0,0,0,0,1,0. Configuración UART: 4800 baudios, 8 bits de datos. Bit de inicio: nivel bajo (0) previo al primer bit de dato. Bit(s) de parada: nivel alto (1) al final de la trama.

Configuración	Tiempo total teó	Tiempo total exp	%
Sin paridad (10 bits: 1+8+1)	2.083 ms	2.100 ms	0.80%
Con paridad, prueba original (11 bits: 1+8+1+1)	2.292 ms	2.300 ms	0.36%
Con paridad, variante modificada (11 bits)	2.292 ms	2.320 ms	1.24%

Tabla 9. Tiempo total de trama para el carácter 'B' (4800 baudios, 8 bits de datos).

Cálculos

$$T_b = \frac{1}{4800} = 208.33 \mu s$$

Sin paridad: nbits = 1 + 8 + 0 + 1 = 10

$$T_{trama} = 10 \times 208.33 = 2.083 ms$$

$$\%E = \frac{|2100.00 - 2083.33|}{2083.33} \times 100 = 0.80 \%$$

Con paridad: nbits = 1 + 8 + 1 + 1 = 11

$$T_{trama} = 11 \times 208.33 = 2.292 ms$$

$$\%error = \frac{|2300.00 - 2291.67|}{2291.67} \times 100 = 0.36 \%$$

$$\%error = \frac{|2320.00 - 2291.67|}{2291.67} \times 100 = 1.24 \%$$

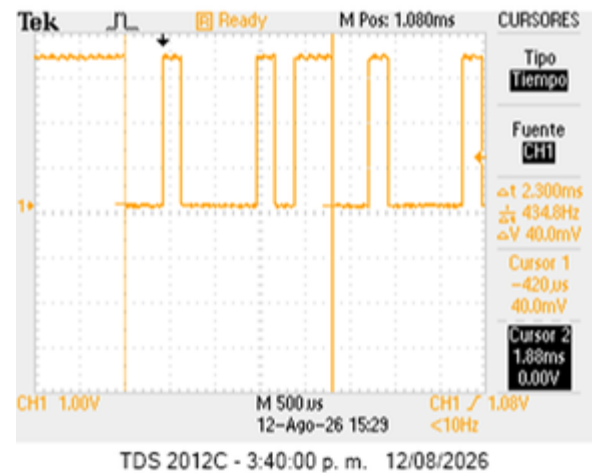


Figure 15 Carácter 'B' con paridad, prueba original ($\Delta t = 2.300 ms$, 4800 baudios).

La trama de B con paridad dura 2.300 ms frente a 2.292 ms teóricos, con 0.36 % de error. La comparación con la captura sin paridad permite observar el incremento temporal de un bit.

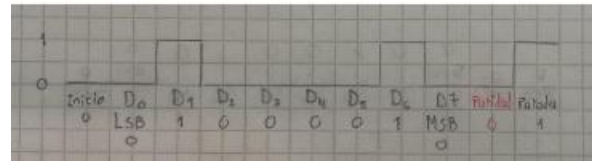


Figure 16. Dibujo Trama "B"

Segmento	Valor	Intervalo teórico
Bit de inicio (start)	0 (bajo)	0.00 μ s – 208.33 μ s
Dato d0 (LSB)	0	208.33 μ s – 416.66 μ s
Dato d1	1	416.66 μ s – 624.99 μ s
Dato d2	0	624.99 μ s – 833.32 μ s
Dato d3	0	833.32 μ s – 1041.65 μ s
Dato d4	0	1041.65 μ s – 1249.98 μ s
Dato d5	0	1249.98 μ s – 1458.31 μ s
Dato d6	1	1458.31 μ s – 1666.64 μ s
Dato d7 (MSB)	0	1666.64 μ s – 1874.97 μ s
Paridad (tipo no confirmado en la captura)	par→0 / impar→1	1874.97 μ s – 2083.30 μ s

Bit de parada (stop)	1 (alto)	2083.30 μ s – 2291.63 μ s
----------------------	----------	-----------------------------------

Tabla 10. Estructura de bits de la trama del carácter 'B' con paridad.

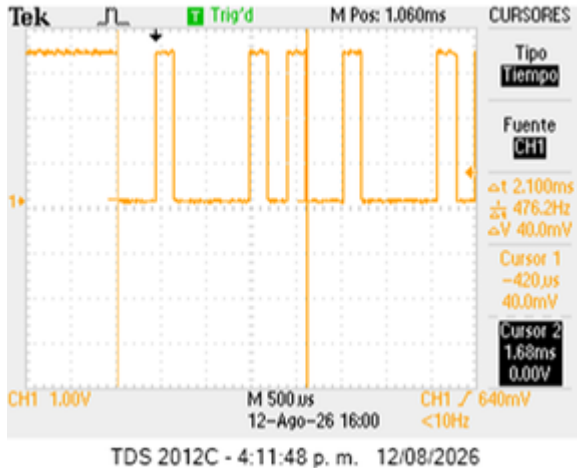


Figure 17. Carácter 'B' sin paridad ($\Delta t = 2.100$ ms, 4800 baudios).

al remover la paridad la trama se reduce a 10 bits; el valor medido (2.100 ms) es un 0.80 % mayor que el teórico (2.083 ms), y la diferencia frente a la Fig. 11 (200 μ s) corresponde a un tiempo de bit completo.

Segmento	Valor	Intervalo teórico
Bit de inicio (start)	0 (bajo)	0.00 μ s – 208.33 μ s
Dato d0 (LSB)	0	208.33 μ s – 416.66 μ s
Dato d1	1	416.66 μ s – 624.99 μ s
Dato d2	0	624.99 μ s – 833.32 μ s
Dato d3	0	833.32 μ s – 1041.65 μ s
Dato d4	0	1041.65 μ s – 1249.98 μ s
Dato d5	0	1249.98 μ s – 1458.31 μ s
Dato d6	1	1458.31 μ s – 1666.64 μ s
Dato d7 (MSB)	0	1666.64 μ s – 1874.97 μ s
Bit de parada (stop)	1 (alto)	1874.97 μ s – 2083.30 μ s

Tabla 11. Estructura de bits de la trama del carácter 'B' sin paridad.

A. Carácter 4: 'v'

Valor decimal: 118. Valor binario (8 bits): 01110110. Orden de transmisión (LSB primero): 0,1,1,0,1,1,1,0. Configuración UART: 9600 baudios, 8 bits de datos. Bit de inicio: nivel bajo (0) previo al primer bit de dato. Bit(s) de parada: nivel alto (1) al final de la trama.

Configuración	Tiempo total teórico	Tiempo total experimental	% error
Sin paridad (10 bits: 1+8+1)	1.0417 ms	1.040 ms	0.16%

Tabla 12. Tiempo total de trama para el carácter 'v' (9600 baudios, 8 bits de datos).

Cálculos:

$$T_b = \frac{1}{9600} = 104.17 \mu s$$

$$\text{Sin paridad: } N_{bits} = 1 + 8 + 0 + 1 = 10$$

$$T_{trama} = 10 \times 104.17 = 1.042 \text{ ms}$$

$$\%E = \frac{|1040.00 - 1041.67|}{1041.67} \times 100 = 0.16 \%$$

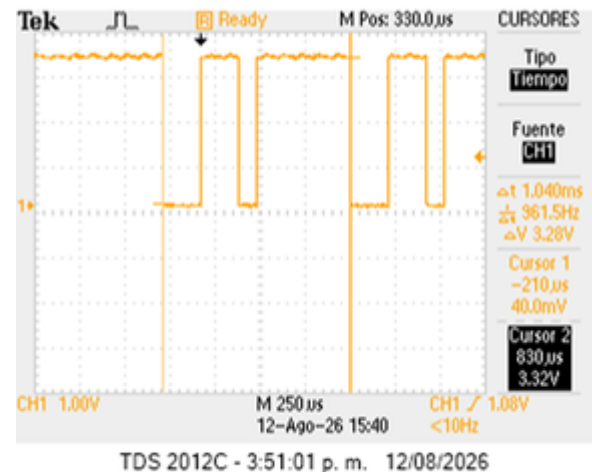


Figure 18. Carácter 'v' sin paridad ($\Delta t = 1.040$ ms, 9600 baudios).

La captura muestra v con 8 bits de datos y sin paridad. La duración de 1.040 ms presenta 0.16 % de error respecto a 1.0417 ms, siendo el mejor ajuste de esta serie.

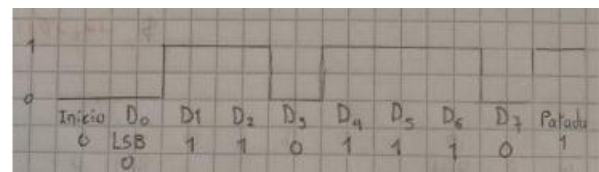


Figure 19. Dibujo Trama "v"

Segmento	Valor	Intervalo teórico
Bit de inicio (start)	0 (bajo)	0.00 μ s – 104.17 μ s
Dato d0 (LSB)	0	104.17 μ s – 208.34 μ s
Dato d1	1	208.34 μ s – 312.51 μ s
Dato d2	1	312.51 μ s – 416.68 μ s
Dato d3	0	416.68 μ s – 520.85 μ s
Dato d4	1	520.85 μ s – 625.02 μ s
Dato d5	1	625.02 μ s – 729.19 μ s
Dato d6	1	729.19 μ s – 833.36 μ s
Dato d7 (MSB)	0	833.36 μ s – 937.53 μ s
Bit de parada (stop)	1 (alto)	937.53 μ s – 1041.70 μ s

Tabla 13. Estructura de bits de la trama del carácter 'v' sin paridad.

A. Carácter 5: '-'

Valor decimal: 45. Valor binario (7 bits): 0101101. Orden de transmisión (LSB primero): 1,0,1,1,0,1,0. Configuración UART: 115200 baudios, 7 bits de datos. Bit de inicio: nivel bajo (0) previo al primer bit de dato. Bit(s) de parada: nivel alto (1) al final de la trama.

Configuración	Tiempo total Teo	Tiempo total Exp	% error
Sin paridad (9 bits: 1+7+1)	78.12 μ s	79.00 μ s	1.12%
Con paridad, prueba original (10 bits: 1+7+1+1)	86.81 μ s	87.60 μ s	0.92%
Con paridad, variante modificada (10 bits)	86.81 μ s	87.00 μ s	0.22%

Tabla 14. Tiempo total de trama para el carácter '-' (115200 baudios, 7 bits de datos).

$$T_{trama} = 9 \times 8.68 = 78.12 \mu s$$

$$\%E = \frac{|79.00 - 78.12|}{78.12} \times 100 = 1.12 \%$$

Con paridad (orig.): $N_{bits} = 1 + 7 + 1 + 1 = 10$

$$T_{trama} = 10 \times 8.68 = 86.81 \mu s$$

$$\%E = \frac{|87.60 - 86.81|}{86.81} \times 100 = 0.92 \%$$

$$\%error = \frac{|87.00 - 86.81|}{86.81} \times 100 = 0.22 \%$$

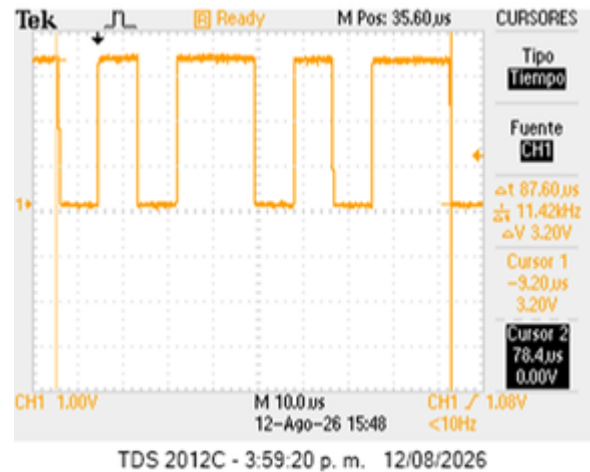


Figure 20 Carácter '-' con paridad, prueba original ($\Delta t = 87.60 \mu$ s, 115 200 baudios).

La trama de - con 7 bits de datos, paridad y parada presenta 87.60 μ s, frente a 86.81 μ s teóricos, con 0.92 % de error. La alta tasa de baudios hace más sensible la medición temporal.

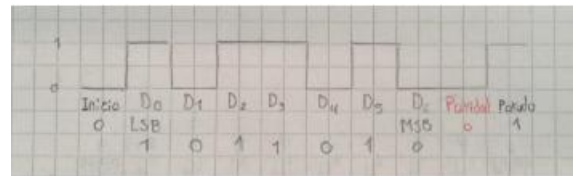


Figure 21. Dibujo Trama "-"

Cálculos

$$T_b = \frac{1}{115200} = 8.68 \mu s$$

Sin paridad: $N_{bits} = 1 + 7 + 0 + 1 = 9$

Segmento	Valor	Intervalo teórico
Bit de inicio (start)	0 (bajo)	0.00 μ s – 8.68 μ s
Dato d0 (LSB)	1	8.68 μ s – 17.36 μ s
Dato d1	0	17.36 μ s – 26.04 μ s
Dato d2	1	26.04 μ s – 34.72 μ s
Dato d3	1	34.72 μ s – 43.40 μ s
Dato d4	0	43.40 μ s – 52.08 μ s
Dato d5	1	52.08 μ s – 60.76 μ s
Dato d6 (MSB)	0	60.76 μ s – 69.44 μ s
Paridad (tipo no confirmado en la captura)	par→0 / impar→1	69.44 μ s – 78.12 μ s
Bit de parada (stop)	1 (alto)	78.12 μ s – 86.80 μ s

Tabla 15. Estructura de bits de la trama del carácter '-' con paridad.

Dato d0 (LSB)	1	8.68 μ s – 17.36 μ s
Dato d1	0	17.36 μ s – 26.04 μ s
Dato d2	1	26.04 μ s – 34.72 μ s
Dato d3	1	34.72 μ s – 43.40 μ s
Dato d4	0	43.40 μ s – 52.08 μ s
Dato d5	1	52.08 μ s – 60.76 μ s
Dato d6 (MSB)	0	60.76 μ s – 69.44 μ s
Bit de parada (stop)	1 (alto)	69.44s – 78.12 μ s

Tabla 16. Estructura de bits de la trama del carácter '-' sin paridad.

VI. MEDICIÓN DEL TIEMPO TOTAL DE LA TRAMA

Se transmitió una trama de 60 caracteres (@) a 600 baudios, 8 bits de datos y paridad par, midiendo el tiempo total de la trama con y sin bit de paridad. Puede ver el código completo (ANEXO D)

Adicionalmente se capturó la señal correspondiente al mensaje 'UMNG LIDER EN INGENIERIA EN TELECOMUNICACIONES' (45 caracteres, incluyendo espacios) a 57 600 baudios, 7 bits de datos, paridad par y 2 bits de parada. Puede ver el código completo en (ANEXO E)

A. Transmisión de 60 caracteres a 600 baudios

Para la configuración de 600 baudios, el tiempo correspondiente a un bit es:

$$T_b = \frac{1}{600} = 1.667 \text{ ms}$$

Con 8 bits de datos, paridad par y un bit de parada, cada carácter está compuesto por:

$$N = 1 + 8 + 1 + 1 = 11 \text{ bits}$$

Por lo tanto, el tiempo teórico de transmisión de un carácter es:

$$T_{\text{Caracter}} = \frac{11}{600} = 18.33 \text{ ms}$$

Como se transmiten 60 caracteres:

$$T_{\text{teo}} = 60 \left(\frac{11}{600} \right) = 1.1 \text{ s}$$

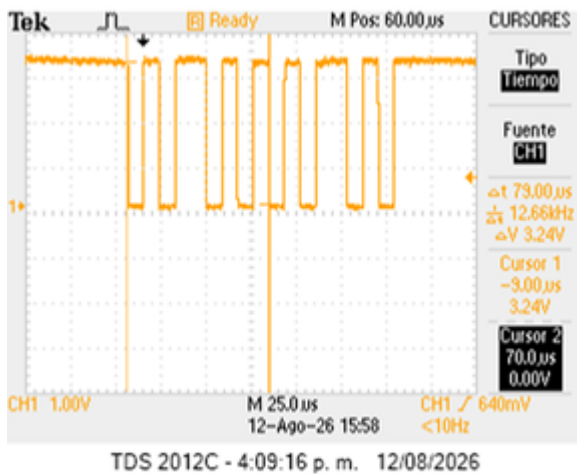


Figure 22 Carácter '-' sin paridad ($\Delta t = 79.00 \mu$ s, 115 200 baudios).

Sin paridad, la duración de 79.00 μ s se compara con 78.12 μ s teóricos y produce 1.12 % de error. La diferencia con la Fig. 14 es cercana a un tiempo de bit.

Segmento	Valor	Intervalo teórico
Bit de inicio (start)	0 (bajo)	0.00 μ s – 8.68 μ s

El tiempo experimental obtenido fue de 1.120 s, por lo que el error porcentual es:

$$\%E = \frac{|1.1 - 1.12|}{1.1} \times 100 = 1.82 \%$$

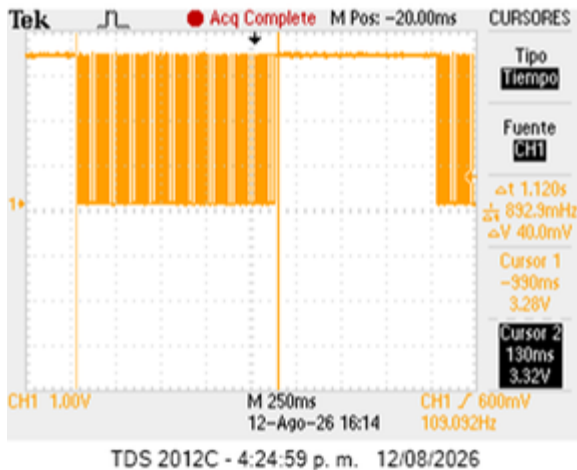


Figure 23. Vista general de la transmisión de 60 caracteres con paridad.

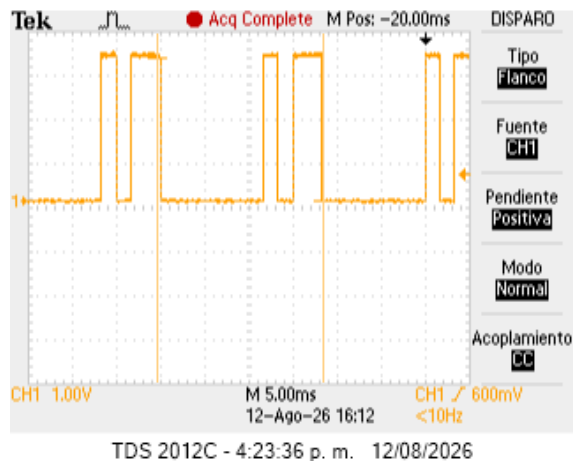


Figure 24. Detalle de un bit en la trama de 60 caracteres con paridad.

Se observa la vista general de la transmisión de los 60 caracteres. La señal corresponde a una secuencia continua de caracteres, donde cada carácter contiene un bit de inicio, ocho bits de datos, un bit de paridad y un bit de parada. En total, cada carácter requiere 11 bits, por lo que los 60 caracteres requieren:

$$N_{total} = 60(11) = 660 \text{ bits}$$

El tiempo experimental de 1.120 s presenta un error del 1.82 % respecto al valor teórico de 1.100 s. La pequeña diferencia

puede atribuirse a la precisión de la medición realizada sobre la captura de la señal.

B. Comparación de 60 caracteres con y sin paridad

Para analizar el efecto de la paridad, se repitió la transmisión de los mismos 60 caracteres a 600 baudios, pero deshabilitando el bit de paridad. En este caso, cada carácter está compuesto por un bit de inicio, ocho bits de datos y un bit de parada:

$$N = 1 + 8 + 1 = 10 \text{ bits}$$

El tiempo teórico de un carácter es:

$$T_{Caracter} = \frac{11}{600} = 16.67 \text{ ms}$$

Para los 60 caracteres:

$$T_{teo} = 60 \left(\frac{10}{600} \right) = 1 \text{ s}$$

El tiempo experimental fue de 1.020 s, con un error de:

$$\%error = \frac{|1.020 - 1.000|}{1.000} \times 100 = 2.00 \%$$

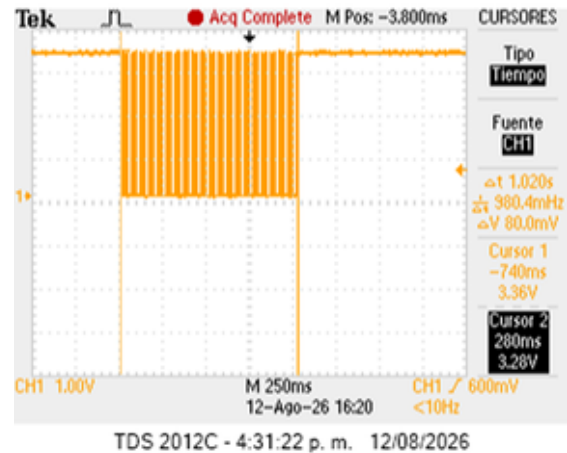


Figure 25. Tiempo total de transmisión de 60 caracteres sin paridad.

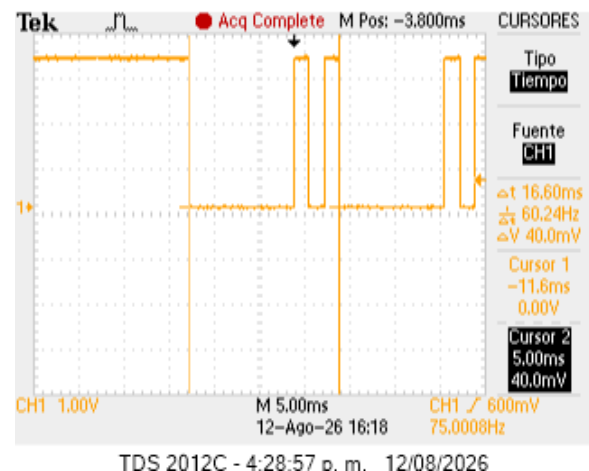


Figure 26. Detalle de un bit en la trama de 60 caracteres sin paridad.

Al comparar ambas configuraciones, con paridad se transmiten 11 bits por carácter, mientras que sin paridad se transmiten 10 bits. Por lo tanto, la eliminación de la paridad reduce un bit por cada carácter:

$$\Delta N = 60(1) = 60 \text{ bits}$$

El tiempo correspondiente a estos 60 bits adicionales es:

$$\Delta T = \frac{600}{60} = 0.100 \text{ s}$$

Por tanto, la diferencia teórica entre ambas transmisiones es de 100 ms. Experimentalmente, la diferencia entre 1.120 s y 1.020 s también es:

$$\Delta T_{exp} = 1.120 - 1.020 = 0.100 \text{ s}$$

Esto confirma experimentalmente que la inclusión de un bit de paridad por carácter aumenta el tiempo total de transmisión en 100 ms para una secuencia de 60 caracteres a 600 baudios.

C. Transmisión del mensaje

Se transmitió el mensaje UMNG LIDER EN INGENIERIA EN TELECOMUNICACIONES, compuesto por 45 caracteres incluyendo los espacios, a una velocidad de 57600 baudios, utilizando 7 bits de datos, paridad par y 2 bits de parada.

Cada carácter está compuesto por:

$$N = 1 + 7 + 1 + 2 = 11 \text{ bits}$$

Por lo tanto, para los 45 caracteres se transmiten:

$$N_{total} = 45(11) = 495 \text{ bits}$$

El tiempo teórico total es:

$$T = \frac{495}{57600} = 8.594 \text{ ms}$$

El tiempo experimental obtenido fue de 8.640 ms. El error porcentual es:

$$\%error = \frac{|8.640 - 8.594|}{8.594} \times 100 = 0.54 \%$$

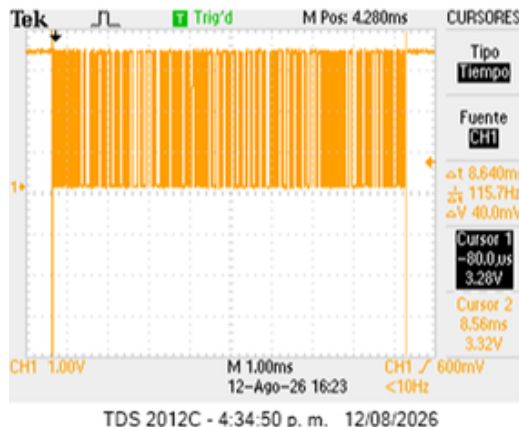


Figure 27. Tiempo total de transmisión del mensaje UMNG

La captura corresponde a la transmisión completa del mensaje utilizando 7 bits de datos, paridad par y 2 bits de parada. El tiempo experimental de 8.640 ms presenta un error de aproximadamente 0.53 % respecto al valor teórico de 8.594 ms.

Este resultado muestra una buena correspondencia entre el tiempo calculado a partir de la configuración UART y el tiempo observado experimentalmente.

D. Comparación de resultados

Los resultados obtenidos permiten comprobar que el tiempo total de transmisión depende directamente de la cantidad de bits que conforman cada carácter y de la velocidad de transmisión.

Prueba	T. teórico	T. experim.	% error
60 caracteres (paridad par)	1.100 s	1.120 s	1.82%
60 caracteres (sin paridad)	1.000 s	1.020 s	2.00%
Mensaje UMNG (45 caract.)	8.59ms	8.640 ms	0.53%

Tabla 17. Comparación del tiempo total de transmisión en las pruebas (60 caracteres y mensaje UMNG).

Para los 60 caracteres a 600 baudios, la inclusión de la paridad aumenta la trama de 10 a 11 bits por carácter, produciendo un incremento teórico de 100 ms en el tiempo total. La medición experimental presentó exactamente la misma diferencia.

En el caso del mensaje de 45 caracteres a 57600 baudios, la configuración de 7 bits de datos, paridad par y 2 bits de parada genera 11 bits por carácter y un tiempo teórico de 8.594 ms. El tiempo experimental de 8.640 ms presentó un error de 0.53 %,

mostrando una buena correspondencia entre el comportamiento teórico y el experimental.

IV. Reto de Programación

A. Interconexión de Hardware

Para la conexión física, se utilizarán los pines GP0 y GP1 de la Raspberry Pi Pico, correspondientes a UART0 TX y UART0 RX, respectivamente (ver Figura 1). Por su parte, en la Raspberry Pi Pico 2W se emplearán los pines GP16 y GP17, correspondientes a UART0 RX y UART0 TX, respectivamente.

De igual forma se conectarán las tierras de ambas placas de desarrollo. Adicionalmente, se conectará una pantalla OLED SSD1306 a la Raspberry Pi Pico 2W mediante el protocolo de comunicación I2C, utilizando los pines GP20 y GP21, correspondientes a I2C0 SDA e I2C0 SCL, respectivamente. La pantalla permitirá visualizar la información recibida a través de la comunicación UART. Físicamente, la conexión entre los dispositivos se presenta de la siguiente manera:

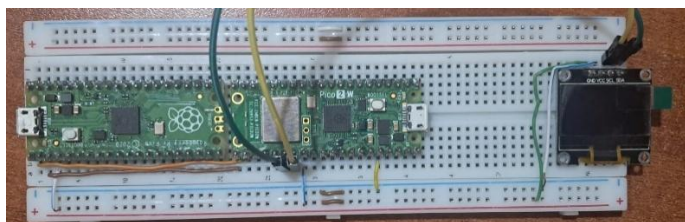


Figure 28. Interconexión de Hardware

B. Transmisión y Recepción de Caracteres (Half-Duplex)

Para la comunicación se configuró la interfaz UART0 con una velocidad de 9600 baudios. La operación del transmisor se desarrolló mediante el programa TX.py. El dato enviado es ingresado exclusivamente mediante la función input() desde la consola de MicroPython. De esta manera, el usuario puede introducir directamente el carácter que desea transmitir, sin utilizar estados lógicos provenientes de los pines físicos como fuente de entrada.

```
TRANSMISOR - PICO
UART: 9600 baudios
TX: GP0
RX: GP1
```

```
Sistema listo
```

```
Ingrese un carácter: H
```

Figure 29. Consola TX

Una vez capturado el carácter, el programa lo transmite mediante UART y permanece a la espera de la respuesta generada por el dispositivo receptor. El código correspondiente se presenta en el ANEXO H.

En el dispositivo receptor se implementó el programa RX.py. Este permanece en espera de información en la interfaz UART y, cuando detecta datos disponibles, procede a recibirlos y decodificarlos. Una vez identificado el carácter enviado, el sistema activa el LED integrado de la Raspberry Pi Pico 2 W durante cinco segundos. Posteriormente, desactiva el indicador y transmite un mensaje de confirmación hacia la Raspberry Pi Pico transmisora mediante el mismo enlace UART.

El código correspondiente se encuentra en el ANEXO H.

```
Sistema listo
```

```
Carácter recibido: H
LED RX encendido durante 5 segundos
LED RX apagado
ACK enviado: ACK:H
Comunicación confirmada
```

Figure 30. Consola RX

La pantalla OLED permite visualizar localmente el carácter recibido y el estado de la comunicación, proporcionando una indicación visual adicional del funcionamiento del sistema.



Figure 31. OLED estado de Espera



Figure 32. OLED estado Procesamiento y LED espera de 5s

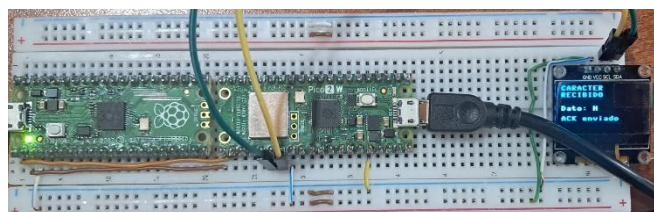


Figure 33. OLED estado Recepción junto con envío ACK y LED confirmación 3s

Una vez que el transmisor recibe correctamente el mensaje de confirmación, activa su LED integrado de manera intermitente durante tres segundos.

Esta acción constituye la confirmación visual de que el carácter enviado fue recibido y procesado correctamente por el segundo microcontrolador. Finalizado este período, el programa retorna automáticamente a la función input (), permitiendo realizar una nueva transmisión sin necesidad de reiniciar el sistema.

Respuesta recibida: ACK:H
ACK CORRECTO
LED TX: parpadeando durante 3 segundos
Comunicación exitosa

Figure 34. ACK Recibido TX

El flujo general de operación implementado puede resumirse de la siguiente manera:

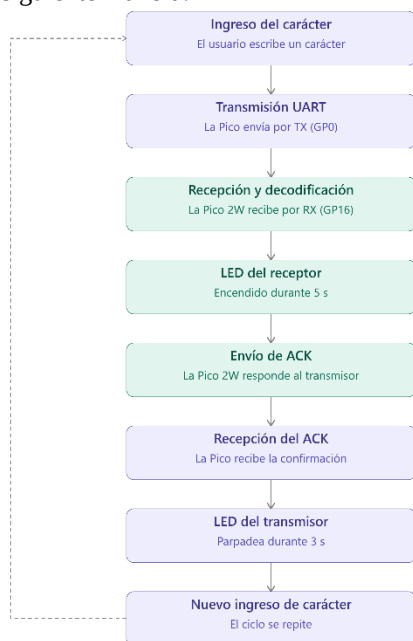


Figure 35. Flujo de la Operación

De esta manera, se obtuvo una comunicación bidireccional con confirmación de recepción. Aunque el intercambio de información se realiza de forma secuencial (primero se transmite el carácter y posteriormente se responde con el ACK), el enlace físico dispone de líneas independientes para TX y RX.

Esto permite realizar el procedimiento de transmisión y confirmación de manera ordenada y controlada.

C. Transmisión de Archivos

Para la segunda etapa se amplió el sistema de comunicación con el propósito de transmitir un archivo de texto completo entre los dos microcontroladores.

La comunicación UART utilizada en el inciso anterior se conservó con la misma configuración de 9600 baudios, el proceso se dividió en tres programas principales: generadortxt.py, enviar.py y recibir.py. Los códigos completos correspondientes a esta etapa se presentan en el **ANEXO I**.

C. 1. Generación del Archivo

El programa generadortxt.py se desarrolló para ejecutarse en la Raspberry Pi Pico transmisora. Su función es generar automáticamente el archivo alfabets.txt, solicitando al usuario mediante consola el número de líneas que desea crear.

El programa restringe la cantidad de líneas a un rango comprendido entre 1 y 1000, verificando además que el valor introducido corresponda a un número entero válido. Para cada línea se genera el alfabeto inglés completo, compuesto por las 26 letras de la A a la Z, acompañado de un identificador numérico de cuatro dígitos. Es decir para cinco líneas se obtiene:

```

0001 ABCDEFGHIJKLMNOPQRSTUVWXYZ
0002 ABCDEFGHIJKLMNOPQRSTUVWXYZ
0003 ABCDEFGHIJKLMNOPQRSTUVWXYZ
0004 ABCDEFGHIJKLMNOPQRSTUVWXYZ
0005 ABCDEFGHIJKLMNOPQRSTUVWXYZ
  
```

V. ANÁLISIS GENERAL DE LOS RESULTADOS

Los resultados experimentales verifican la relación $t_b = 1/B$. Al aumentar los baudios, el tiempo de bit disminuye y las mediciones se mantienen próximas a los valores teóricos. El error del barrido temporal varió entre 0.32 % y 1.38 %.

En los caracteres ?, B y - se comprobó experimentalmente que retirar el bit de paridad reduce la duración de la trama en aproximadamente un tiempo de bit. El cambio del tipo de paridad no modifica el número de bits transmitidos, por lo que su efecto es principalmente lógico y no temporal.

En la transmisión de 60 caracteres a 600 baudios, la configuración con paridad utiliza 660 bits y la configuración sin paridad 600 bits; por ello la diferencia teórica es de 100 ms, valor que se reproduce en la medición. El mensaje de 45 caracteres a 57600 baudios, con 7 bits de datos, paridad par y dos bits de parada, presentó 0.53 % de error.

Los niveles de tensión se mantuvieron constantes en aproximadamente 3.40 V máximo y 0.00 V mínimo. Por tanto, el cambio de baudios modificó principalmente la escala temporal de la señal y no sus niveles lógicos.

V. ANÁLISIS GENERAL DE LOS RESULTADOS

Los resultados experimentales verifican la relación $t_b = 1/\text{baudrate}$. Al aumentar los baudios, el tiempo de bit disminuye y las mediciones se mantienen próximas a los valores teóricos. El error del barrido temporal varió entre 0.32 % y 1.38 %.

En los caracteres ?, B y - se comprobó experimentalmente que retirar el bit de paridad reduce la duración de la trama en aproximadamente un tiempo de bit. El cambio del tipo de paridad no modifica el número de bits transmitidos, por lo que su efecto es principalmente lógico y no temporal.

En la transmisión de 60 caracteres a 600 baudios, la configuración con paridad utiliza 660 bits y la configuración sin

paridad 600 bits; por ello la diferencia teórica es de 100 ms, valor que se reproduce en la medición. El mensaje de 45 caracteres a 57600 baudios, con 7 bits de datos, paridad par y dos bits de parada, presentó 0.53 % de error.

Los niveles de tensión se mantuvieron constantes en aproximadamente 3.40 V máximo y 0.00 V mínimo. Por tanto, el cambio de baudios modificó principalmente la escala temporal de la señal y no sus niveles lógicos.

VII. CONCLUSIONES

- Se verificó experimentalmente la relación de tiempo de transmisión de bits en un rango de seis tasas de transmisión (1200 a 115200 baudios), obteniendo errores porcentuales entre 0.32 % y 1.38 %. Esto confirma que la UART de la Raspberry Pi Pico 2W genera tiempos de bit consistentes con la configuración establecida por software, dentro del margen de tolerancia esperado para el oscilador interno del dispositivo y la resolución de medición del osciloscopio.
- El análisis de cinco caracteres ASCII transmitidos bajo distintas combinaciones de baudios, bits de datos y paridad permitió comprobar que la estructura de la trama serial (bit de inicio, datos ordenados de LSB a MSB, paridad opcional y bit(s) de parada), donde se conserva de forma consistente en la señal física observada. Los errores de tiempo total de trama no superaron el 1.60 % en ninguno de los casos evaluados.
- Se cuantificó el efecto del bit de paridad sobre la duración de la trama: su inclusión añade sistemáticamente un tiempo de bit adicional a la transmisión, mientras que el tipo de paridad (par o impar) no genera diferencias apreciables en la temporización, dado que ambas configuraciones ocupan el mismo número de bits.
- El incremento de la tasa de baudios reduce proporcionalmente el tiempo de transmisión, pero incrementa la sensibilidad relativa de la medición frente a errores de resolución temporal. Esto se evidenció en el mayor error porcentual obtenido a 115200 baudios en comparación con el registrado a 1200 baudios.
- La prueba de trama extendida (60 caracteres a 600 baudios) mostró que la acumulación de bits en tramas largas amplifica el error relativo respecto a mediciones de un único carácter. Por su parte, la transmisión del mensaje "UMNG LIDER EN INGENIERIA EN TELECOMUNICACIONES" (45 caracteres, 57600 baudios, 7 bits de datos, paridad par, 2 bits de parada) validó con alta precisión (0.53 % de error) una configuración de trama de mayor complejidad.

VIII. REFERENCIAS

[1] Rugeles Uribe, J. de J. Guía de laboratorio: "Explorando la comunicación RS232". Comunicaciones Digitales, Universidad Militar Nueva Granada, 2026.

[2] MicroPython Documentation. machine.UART — UART class. <https://docs.micropython.org/en/latest/library/machine.UART.html>

[3] ANSI. American Standard Code for Information Interchange (ASCII), ANSI X3.4.

[4] National Institute of Standards and Technology (NIST), *ASCII: American Standard Code for Information Interchange*, U.S. Department of Commerce. [En línea]. Disponible en: <https://www.nist.gov/>

[5] Raspberry Pi Ltd., "Raspberry Pi Pico Python SDK," *Raspberry Pi Documentation*. [En línea]. Disponible en: <https://www.raspberrypi.com/documentation/microcontrollers/python.html>

VIII. ANEXOS

Anexo A. Código base MicroPython empleado para la transmisión de caracteres (rs232.py), suministrado por la guía de laboratorio, modificado para variar baudios, bits de datos y paridad según cada prueba.

```
1 import machine
2 import utime
3 from machine import Pin, UART
4
5 led = machine.Pin("LED", machine.Pin.OUT)
6 uart = UART(0, baudrate=9600, bits=8, parity=0, tx=Pin(0), rx=Pin(1))
7
8 while True:
9     led.on()
10    uart.write("U")
11    utime.sleep(1)
12    led.off()
13    utime.sleep(1)
```

Anexo B Medición tiempo de bit (Diferentes baudios)

B.1 — Barrido de baudios (Tabla 2, carácter 'Y')

```
1 import machine
2 import utime
3 from machine import Pin, UART
4
5 led = machine.Pin("LED", machine.Pin.OUT)
6 # baudrate modificado en cada prueba: 1200 / 4800 / 9600 / 38400 / 57600 / 115200
7 uart = UART(0, baudrate=115200, bits=8, parity=None, tx=Pin(0), rx=Pin(1))
8
9 while True:
10     led.on()
11     uart.write("Y")
12     utime.sleep(1)
13     led.off()
14     utime.sleep(1)
```

ANEXO C Caracteres

C.1 — Carácter '?' (1200 baudios, 7 bits) — Tabla 3, fila 1

```
1 import machine
2 import utime
3 from machine import Pin, UART
4
5 led = machine.Pin("LED", machine.Pin.OUT)
6 # parity=None -> prueba sin paridad
7 # parity=0 -> con paridad par (prueba original)
8 # parity=1 -> con paridad impar (variante modificada)
9 uart = UART(0, baudrate=1200, bits=7, parity=0, tx=Pin(0), rx=Pin(1))
10
11 while True:
12     led.on()
13     uart.write("?")
14     utime.sleep(1)
15     led.off()
16     utime.sleep(1)
```

C.2 — Carácter '\$' (57600 baudios, 8 bits, con paridad) — Tabla 3, fila 2

```
1 import machine
2 import utime
3 from machine import Pin, UART
4
5 led = machine.Pin("LED", machine.Pin.OUT)
6 uart = UART(0, baudrate=57600, bits=8, parity=0, tx=Pin(0), rx=Pin(1))
7
8 while True:
9     led.on()
10    uart.write("$")
11    utime.sleep(1)
12    led.off()
13    utime.sleep(1)
```

C.3 — Carácter 'B' (4800 baudios, 8 bits) — Tabla 3, fila 3

```
1 import machine
2 import utime
3 from machine import Pin, UART
4
5 led = machine.Pin("LED", machine.Pin.OUT)
6 # parity=None -> sin paridad
7 # parity=0 -> con paridad par (prueba original)
8 # parity=1 -> con paridad impar (variante modificada)
9 uart = UART(0, baudrate=4800, bits=8, parity=0, tx=Pin(0), rx=Pin(1))
10
11 while True:
12     led.on()
13     uart.write("B")
14     utime.sleep(1)
15     led.off()
16     utime.sleep(1)
```

C.4 — Carácter 'v' (9600 baudios, 8 bits, sin paridad) — Tabla 3, fila 4

```
1 import machine
2 import utime
3 from machine import Pin, UART
4
5 led = machine.Pin("LED", machine.Pin.OUT)
6 uart = UART(0, baudrate=9600, bits=8, parity=None, tx=Pin(0), rx=Pin(1))
7
8 while True:
9     led.on()
10    uart.write("v")
11    utime.sleep(1)
12    led.off()
13    utime.sleep(1)
```

C.5 — Carácter '-' (115200 baudios, 7 bits) — Tabla 3, fila 5

```
1 import machine
2 import utime
3 from machine import Pin, UART
4
5 led = machine.Pin("LED", machine.Pin.OUT)
6 # parity=None -> sin paridad
7 # parity=0 -> con paridad par (prueba original)
8 # parity=1 -> con paridad impar (variante modificada)
9 uart = UART(0, baudrate=115200, bits=7, parity=0, tx=Pin(0), rx=Pin(1))
10
11 while True:
12     led.on()
13     uart.write("-")
14     utime.sleep(1)
15     led.off()
16     utime.sleep(1)
```

ANEXO D Transmisión 60 caracteres

D.1 — Trama de 60 caracteres (600 baudios, 8 bits) — con y sin paridad

```
1 import machine
2 import utime
3 from machine import Pin, UART
4 led = machine.Pin("LED", machine.Pin.OUT)
5 # parity=0 -> con paridad par (660 bits totales)
6 # parity=None -> sin paridad (600 bits totales)
7 uart = UART(0, baudrate=600, bits=8, parity=0, tx=Pin(0), rx=Pin(1))
8
9 trama = "A" * 60 # secuencia de 60 caracteres ASCII
10
11 while True:
12     led.on()
13     uart.write(trama)
14     utime.sleep(2)
15     led.off()
16     utime.sleep(2)
```

ANEXO E Transmisión Mensaje UMNG

E.1 — Mensaje UMNG (57600 baudios, 7 bits, paridad par, 2 bits de parada)

```
1 import machine
2 import utime
3 from machine import Pin, UART
4
5 led = machine.Pin("LED", machine.Pin.OUT)
6 uart = UART(0, baudrate=57600, bits=7, parity=0, stop=2, tx=Pin(0), rx=Pin(1))
7
8 mensaje = "UMNG LIDER EN INGENIERIA EN TELECOMUNICACIONES"
9
10 while True:
11     led.on()
12     uart.write(mensaje)
13     utime.sleep(2)
14     led.off()
15     utime.sleep(2)
```

Anexo G. Tabla de equivalencias ASCII de los caracteres empleados

Carácter	Valor Decimal	Valor Binario (8 bits)
?	63	111111
\$	36	100100
B	66	1000010
v	118	11101110
-	45	101101
U	85	1010101
Y	89	1011001
@	64	1000000

Anexo H. Códigos de Half-Duplex

H. 1 — TX.py

```
from machine import UART, Pin
import time
# CONFIGURACIÓN UART
uart = UART(
    0,
    baudrate=9600,
    tx=Pin(0),
    rx=Pin(1),
    timeout=1000,
    timeout_char=100
)
```



```
# LED integrado
led = Pin("LED", Pin.OUT)
print("TRANSMISOR - PICO")
print("UART: 9600 baudios")
print("TX: GP0")
print("RX: GP1")
print()
print("Sistema listo")
print()
while True:
    # SOLICITAR CARÁCTER
    mensaje = input("Ingrese un carácter: ")
    if len(mensaje) != 1:
        print("ERROR: ingrese solamente un carácter.")
        print()
        continue
    # TRANSMITIR
    uart.write(mensaje + "\n")
    print("Enviado:", mensaje)
    # ESPERAR ACK
    ack_recibido = False
    tiempo_inicio = time.ticks_ms()
    while True:
        if uart.any():
            respuesta = uart.readline()
            if respuesta:
                respuesta = respuesta.decode().strip()
                print("Respuesta recibida:", respuesta)
                # ACK esperado
                if respuesta == "ACK:" + mensaje:
                    ack_recibido = True
                    print("ACK CORRECTO")
                else:
                    print("ACK INCORRECTO")
                    break
            if time.ticks_diff(
                time.ticks_ms(),
                tiempo_inicio
            ) > 8000:
                print("TIEMPO DE ESPERA AGOTADO")
                break
    # LED TX
    if ack_recibido:
        print("LED TX: parpadeando durante 3 segundos")
        tiempo_inicio = time.ticks_ms()
        while time.ticks_diff(
            time.ticks_ms(),
            tiempo_inicio
        ) < 3000:
            led.on()
            time.sleep(0.25)
            led.off()
            time.sleep(0.25)
        led.off()
        print("Comunicación exitosa")
    else:
        print("No se pudo confirmar el carácter")
    print()
```

H. 2 – RX.py

```
from machine import UART, Pin, I2C
import ssd1306
import time
```

```
# CONFIGURACIÓN UART
uart = UART(
    0,
    baudrate=9600,
    tx=Pin(16),
    rx=Pin(17),
    timeout=1000,
    timeout_char=100
)
# LED
led = Pin("LED", Pin.OUT)
# OLED SSD1306
i2c = I2C(
    0,
    scl=Pin(21),
    sda=Pin(20),
    freq=400000
)
oled = ssd1306.SSD1306_I2C(
    128,
    64,
    i2c
)
# PANTALLA INICIAL
oled.fill(0)
oled.text("RECEPTOR", 0, 0)
oled.text("PICO 2 W", 0, 12)
oled.text("Esperando...", 0, 30)
oled.text("UART listo", 0, 45)
oled.show()
# CONSOLA
print("RECEPTOR - PICO 2 W")
print("UART: 9600 baudios")
print("RX: GP17")
print("TX: GP16")
print("OLED:")
print("SDA: GP20")
print("SCL: GP21")
print()
print("Sistema listo")
print()
# RECEPCIÓN
while True:
    if uart.any():
        dato = uart.readline()
        if dato:
            mensaje = dato.decode().strip()
            if len(mensaje) == 0:
                continue
            # MOSTRAR EN CONSOLA
            print("Carácter recibido:", mensaje)
            # MOSTRAR EN OLED
            oled.fill(0)
            oled.text("CARACTER", 0, 0)
            oled.text("RECIBIDO", 0, 10)
            oled.text(
                "Dato: " + mensaje,
                0,
                30
            )
            oled.text(
                "Procesando...",
                0,
                45
            )
```



```
)
oled.show()
# LED RX - 5 SEGUNDOS
print("LED RX encendido durante 5 segundos")
led.on()
time.sleep(5)
led.off()
print("LED RX apagado")
# ENVIAR ACK
respuesta = "ACK:" + mensaje
uart.write(respuesta + "\n")
print(
    "ACK enviado:",
    respuesta
)
# OLED - CONFIRMACIÓN
oled.fill(0)
oled.text("CARACTER", 0, 0)
oled.text("RECIBIDO", 0, 10)
oled.text(
    "Dato: " + mensaje,
    0,
    30
)
oled.text(
    "ACK enviado",
    0,
    45
)
oled.show()
print("Comunicación confirmada")
print()
```

Anexo I. Códigos de Transmisión de Archivos

I. 1 - generadortxt.py

```
print("    GENERADOR DE ALFABETOS")
# Alfabeto inglés
alfabeto = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
while True:
    try:
        cantidad = int(input("¿Cuántas líneas desea generar? (1-1000):
    "))
        if 1 <= cantidad <= 1000:
            break
        print("ERROR: debe ingresar un número entre 1 y 1000.")
    except ValueError:
        print("ERROR: debe ingresar un número entero.")
# Crear archivo
with open("alfabetos.txt", "w") as archivo:
    for numero in range(1, cantidad + 1):
        linea = "{:04d} {} \n".format(numero, alfabeto)
        archivo.write(linea)
print()
print("Archivo alfabetos.txt creado correctamente.")
print("Número de líneas:", cantidad)
print()
```

I. 2 - enviar.py

```
from machine import UART, Pin
import time
```

```
# CONFIGURACIÓN UART
uart = UART(
    0,
    baudrate=9600,
    tx=Pin(0),
    rx=Pin(1),
    timeout=1000,
    timeout_char=100
)
print("    ENVÍO Y MEDICIÓN DE ARCHIVO")
print("UART: 9600 baudios")
print("TX: GP0")
print("RX: GP1")
print()
# INTERVALO
while True:
    try:
        intervalo = float(
            input("Intervalo entre líneas (segundos): ")
        )
        if intervalo >= 0:
            break
        print("ERROR: el intervalo no puede ser negativo.")
    except ValueError:
        print("ERROR: introduzca un número válido.")
# ABRIR ARCHIVO
try:
    archivo = open("alfabetos.txt", "r")
except OSError:
    print("ERROR: no se encontró alfabetos.txt")
    print("Ejecute primero generadortxt.py.")
    while True:
        pass
# INICIO DE LA TRANSFERENCIA
contador = 0
print()
print("Preparando transmisión...")
time.sleep(1)
print("INICIO DE LA TRANSFERENCIA")
print()
tiempo_inicio = time.ticks_ms()
# TRANSMISIÓN
for linea in archivo:
    linea = linea.strip()
    if len(linea) == 0:
        continue
    contador += 1
    # Enviar línea
    uart.write(linea + "\n")
    print("Enviando línea", contador)
    print(linea)
    # Esperar ACK
    ack_recibido = False
    tiempo_ack = time.ticks_ms()
    while True:
        if uart.any():
            respuesta = uart.readline()
            if respuesta:
                respuesta = respuesta.decode().strip()
                if respuesta == "ACK":
                    ack_recibido = True
                    print("ACK recibido")
                else:
                    print("Respuesta:", respuesta)
```

```

        break
    # Timeout
    if time.ticks_diff(
        time.ticks_ms(),
        tiempo_ack
    ) > 10000:
        print("ERROR: timeout esperando ACK")
        break
    # Si no se recibió ACK
    if not ack_recibido:
        print()
        print("TRANSFERENCIA INTERRUMPIDA")
        break
    # Intervalo entre líneas
    if intervalo > 0:
        time.sleep(intervalo)
# FINALIZACIÓN
archivo.close()
tiempo_fin = time.ticks_ms()
tiempo_total_ms = time.ticks_diff(
    tiempo_fin,
    tiempo_inicio
)
tiempo_total_s = tiempo_total_ms / 1000
# RESULTADOS
print()
print("    TRANSFERENCIA FINALIZADA")
print("Líneas enviadas:", contador)
print("Intervalo:", intervalo, "segundos")
print("Tiempo total:", tiempo_total_s, "segundos")
if contador > 0:
    promedio = tiempo_total_s / contador
    print(
        "Tiempo promedio por línea:",
        promedio,
        "segundos"
    )

```

I. 3 - recibir.py

```

from machine import UART, Pin, I2C
import ssd1306
import time
# UART
uart = UART(
    0,
    baudrate=9600,
    tx=Pin(16),
    rx=Pin(17),
    timeout=1000,
    timeout_char=100
)
# LED
led = Pin("LED", Pin.OUT)
i2c = I2C(
    0,
    scl=Pin(21),
    sda=Pin(20),
    freq=400000
)
oled = ssd1306.SSD1306_I2C(
    128,
    64,
    i2c
)

```

```

# FUNCIÓN PARA ACTUALIZAR OLED
def mostrar_oled(linea_numero, total, estado):
    oled.fill(0)
    oled.text("RECIBIENDO", 0, 0)
    oled.text("ARCHIVO", 0, 10)
    oled.text("Linea: {:04d}".format(linea_numero), 0, 25)
    oled.text(
        "Recibidas: {}".format(total),
        0,
        37
    )
    oled.text(
        "Estado: " + estado,
        0,
        52
    )
    oled.show()
# PANTALLA INICIAL
oled.fill(0)
oled.text("RECEPTOR", 0, 0)
oled.text("PICO 2 W", 0, 12)
oled.text("Esperando...", 0, 30)
oled.text("UART listo", 0, 45)
oled.show()
# CONSOLA
print("    RECEPTOR DE ARCHIVO")
print("UART: 9600 baudios")
print("RX: GP17")
print("TX: GP16")
print("OLED: GP20/GP21")
print()
print("Esperando archivo...")
print()
# CREAR ARCHIVO
archivo = open("recibido.txt", "w")
contador = 0
# RECEPCIÓN
while True:
    if uart.any():
        dato = uart.readline()
        if dato:
            linea = dato.decode().strip()
            if len(linea) > 0:
                contador += 1
                print("Línea recibida:", linea)
                # GUARDAR EN ARCHIVO
                archivo.write(linea + "\n")
                archivo.flush()
                # LED
                led.on()
                time.sleep(0.1)
                led.off()
                # OLED
                # Intentamos extraer el número
                try:
                    numero = int(linea[:4])
                except:
                    numero = contador
                mostrar_oled(
                    numero,
                    contador,
                    "OK"
                )
            # ACK

```



```
uart.write("ACK\n")  
print("ACK enviado")  
print("Líneas recibidas:", contador)  
print()
```